

Schema

Vincoli

PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK e DEFAULT spiegati in modo pratico.

Regole che proteggono i dati

I **vincoli** sono regole che il database controlla prima di salvare un dato.

Non servono a complicare la vita. Servono a evitare dati rotti, duplicati o incoerenti.

È come avere qualcuno all'ingresso di un magazzino che controlla: "questo pacco ha l'etichetta?", "questo codice esiste già?", "questo valore ha senso?".

I vincoli più comuni

Ecco quelli che incontrerai più spesso:

Vincolo	A cosa serve
PRIMARY KEY	identifica ogni riga
FOREIGN KEY	collega una tabella a un'altra
UNIQUE	impedisce valori duplicati
NOT NULL	rende una colonna obbligatoria
CHECK	controlla una regola personalizzata
DEFAULT	assegna un valore se non lo specifichi

Un esempio completo

Questa tabella usa più vincoli insieme:

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  nome VARCHAR(100) NOT NULL,  
  eta INT CHECK (eta >= 0),  
  attivo BOOLEAN DEFAULT TRUE  
);
```

Leggiamola pezzo per pezzo:

- `id INT PRIMARY KEY` : ogni cliente ha un identificatore unico.
- `email VARCHAR(150) UNIQUE NOT NULL` : l'email è obbligatoria e non può essere duplicata.
- `nome VARCHAR(100) NOT NULL` : il nome è obbligatorio.
- `eta INT CHECK (eta >= 0)` : l'età non può essere negativa.
- `attivo BOOLEAN DEFAULT TRUE` : se non dici nulla, il cliente parte come attivo.

Perché sono utili

Senza vincoli, il database potrebbe accettare dati sbagliati:

- due utenti con la stessa email
- un prodotto con prezzo negativo
- un cliente senza nome
- un ordine collegato a un cliente che non esiste

I controlli nel codice dell'applicazione sono utili, ma non bastano sempre. Il database è l'ultimo posto in cui puoi fermare un dato sbagliato prima che venga salvato.

Un esempio con CHECK

`CHECK` ti permette di scrivere una regola semplice.

```
prezzo DECIMAL(10, 2) CHECK (prezzo >= 0)
```

Questa regola dice: il prezzo deve essere maggiore o uguale a zero.

Se qualcuno prova a inserire un prezzo negativo, il database rifiuta il dato.

Un esempio con **DEFAULT**

DEFAULT entra in gioco quando non specifichi un valore.

```
attivo BOOLEAN DEFAULT TRUE
```

Se inserisci un cliente senza dire se è attivo, il database userà **TRUE**.

È come dire: “se non ricevi istruzioni diverse, parti da questo valore”.

Riepilogo rapido

- I vincoli sono regole applicate dal database.
- **PRIMARY KEY** identifica le righe.
- **FOREIGN KEY** protegge i collegamenti tra tabelle.
- **UNIQUE** evita duplicati.
- **NOT NULL**, **CHECK** e **DEFAULT** aiutano a mantenere dati più puliti.

Modellazione dati

Come progettare uno schema ordinato prima ancora di iniziare a riempirlo.

Prima di scrivere query, conviene disegnare la mappa

La modellazione dati è il momento in cui decidi come organizzare il mondo dentro il database. Prima di scrivere query, conviene fermarsi a capire quali oggetti esistono e come si collegano.

È come disegnare la pianta di una casa prima di iniziare i lavori.

Decidi:

- quali tabelle servono
- quali colonne deve avere ogni tabella
- come si collegano tra loro

:::note Uno schema è il progetto del database. Pensa allo schema come alla mappa dell'armadio: ti dice quali ripiani esistono e cosa va messo in ognuno. :::

Perché questa fase conta così tanto

Uno schema chiaro rende più semplice:

- scrivere query corrette
- evitare duplicazioni inutili
- capire dove deve vivere ogni dato
- far crescere il progetto senza caos

Un piccolo esempio mentale

Se stai costruendo un e-commerce, potresti identificare entità come:

- clienti
- prodotti
- ordini
- righe ordine

Ogni tabella dovrebbe rappresentare un concetto chiaro. Se una tabella prova a fare troppe cose insieme, di solito è un campanello d'allarme.

Una domanda pratica per iniziare

Quando non sai da dove partire, chiediti:

1. quali cose devo ricordare?
2. quali informazioni descrivono ogni cosa?
3. quali cose sono collegate tra loro?

Se stai progettando una biblioteca, le cose potrebbero essere `libri`, `lettori` e `prestiti`.

Un esempio piccolo

Questa tabella rappresenta un solo concetto: i clienti.

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100),  
  email VARCHAR(150)  
);
```

La tabella non contiene ordini, pagamenti o prodotti. **Contiene solo informazioni sui clienti.**

Questa separazione rende il database più facile da capire e da far crescere.

Il segnale che qualcosa non va

Fermati se una colonna sembra contenere una lista intera, per esempio `prodotti_comprati`, oppure se ripeti sempre gli stessi dati in tante righe.

Di solito è il database che ti sta dicendo: **"forse qui serve una tabella in più"**.

Sequence e auto-increment

Come generare identificatori automatici senza sceglierli a mano.

Dare numeri a mano è scomodo e rischioso

Molte tabelle hanno una colonna `id` che cresce automaticamente. È come il numeretto preso in fila: ogni volta il sistema assegna il successivo.

Questo evita di dover scegliere gli identificatori a mano, con il rischio di errori o duplicati.

Un esempio semplice

In molti database puoi chiedere al sistema di creare automaticamente l'identificatore:

```
CREATE TABLE clienti (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nome VARCHAR(100)  
);
```

Quando inserisci un cliente, non scrivi tu l' `id` :

```
INSERT INTO clienti (nome)  
VALUES ('Luca');
```

Il database assegna il prossimo numero disponibile.

I nomi cambiano a seconda del database:

- PostgreSQL: `IDENTITY` o `SERIAL`
- MySQL: `AUTO_INCREMENT`
- SQL Server: `IDENTITY`
- SQLite: `INTEGER PRIMARY KEY`

Perché è comodo

L'auto-incremento è utile quando:

- hai bisogno di una chiave primaria numerica
- vuoi evitare collisioni tra identificatori
- non vuoi preoccuparti di scegliere il prossimo numero

Una cosa importante da capire

Il numero automatico serve soprattutto come identificatore tecnico. Non sempre è una buona idea attribuirgli significati di business, come numero fattura o codice pubblico visibile all'utente.

Per quei casi, spesso servono regole diverse.

:::note Se vedi salti nei numeri, non è per forza un problema. Gli identificatori automatici servono a distinguere le righe, non a contare perfettamente senza buchi. :::

Relazioni tra tabelle

Uno a uno, uno a molti e molti a molti spiegati con esempi concreti.

Quando le tabelle si collegano

Un database diventa davvero utile quando le tabelle non restano isolate.

Pensa a una scuola. Hai studenti, classi, insegnanti e voti. Ogni elenco ha senso da solo, ma il valore vero arriva quando capisci come si collegano.

In SQL questi collegamenti si chiamano **relazioni**.

Le relazioni più comuni sono tre:

- uno a uno
- uno a molti
- molti a molti

Uno a uno

Una relazione uno a uno significa che una riga di una tabella corrisponde a una sola riga di un'altra tabella.

Esempio: un utente può avere un solo profilo esteso.

È come una persona e la sua carta d'identità: una persona ha una carta, e quella carta appartiene a quella persona.

```
CREATE TABLE utenti (  
  id INT PRIMARY KEY,  
  email VARCHAR(150) NOT NULL  
);  
  
CREATE TABLE profili (  
  utente_id INT PRIMARY KEY,  
  bio VARCHAR(500),  
  FOREIGN KEY (utente_id) REFERENCES utenti(id)  
);
```

Qui `profili.utente_id` identifica il profilo e, allo stesso tempo, punta all'utente collegato.

Uno a molti

Questa è una delle relazioni più comuni.

Un cliente può avere molti ordini. Ma ogni ordine appartiene a un solo cliente.

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE ordini (  
  id INT PRIMARY KEY,  
  cliente_id INT NOT NULL,  
  totale DECIMAL(10, 2) NOT NULL,  
  FOREIGN KEY (cliente_id) REFERENCES clienti(id)  
);
```

La colonna `cliente_id` sta nella tabella `ordini`, perché ogni ordine deve ricordare a quale cliente appartiene.

È come una biblioteca: un lettore può fare molti prestiti, ma ogni prestito è intestato a un lettore preciso.

Molti a molti

Una relazione molti a molti significa che entrambe le parti possono collegarsi a più righe.

Per esempio:

- un ordine può contenere molti prodotti
- lo stesso prodotto può comparire in molti ordini

In questi casi si usa una **tabella ponte**.

```
CREATE TABLE righe_ordine (  
  ordine_id INT,  
  prodotto_id INT,  
  quantita INT,  
  PRIMARY KEY (ordine_id, prodotto_id),  
  FOREIGN KEY (ordine_id) REFERENCES ordini(id),  
  FOREIGN KEY (prodotto_id) REFERENCES prodotti(id)  
);
```


`righe_ordine` collega ordini e prodotti. In più salva anche la quantità.

È come un foglio di iscrizioni: non è solo la lista degli studenti e non è solo la lista dei corsi. Dice quale studente partecipa a quale corso.

Come capire che relazione ti serve

Fai la domanda in entrambe le direzioni.

Per esempio:

- un cliente può avere molti ordini?
- un ordine può appartenere a molti clienti?

Se "molti" vale da una parte sola, probabilmente hai una relazione uno a molti.

Se "molti" vale da entrambe le parti, probabilmente ti serve una tabella ponte.

Il ruolo delle chiavi esterne

Una **chiave esterna** (`FOREIGN KEY`) protegge il collegamento tra due tabelle.

Senza questa regola, potresti salvare un ordine con `cliente_id = 999` anche se non esiste nessun cliente con id 999.

Con la chiave esterna, il database controlla per te che il collegamento abbia senso.

Riepilogo rapido

- Uno a uno: una riga corrisponde a una sola riga.
- Uno a molti: una riga può collegarsi a molte righe.
- Molti a molti: di solito serve una tabella ponte.
- Le chiavi esterne aiutano il database a proteggere i collegamenti.